Campus Lost and Found System

A Centralized Web Platform for Reporting, Searching, and Claiming Lost Items

Project Abstract

Tech Stack: React.js • Spring Boot • MySQL • JWT Security

Name	G SriVidya Reddy
Roll No.	24EG105P29
Branch	CSE - P

1. Problem Statement

In large college campuses, students and staff frequently misplace personal belongings such as ID cards, electronics, keys, and books. The traditional approach to recovering these items relies heavily on fragmented WhatsApp groups, physical notice boards, or depositing items at scattered security desks. This unstructured process leads to several recurring problems:

- **Inefficiency** — It takes too long to find the right person to return an item to.
- **Lack of Visibility** — Lost-and-found posts get buried in chat groups and are easily missed.
- **Security Risks** — There is no reliable way to verify the true owner of an item before handing it over.
- **No Centralization** — There is no single, dependable platform serving the entire campus.

2. Proposed Solution

We propose a centralized web-based **Lost and Found Platform** built specifically for the campus community. The system allows students and staff to:

- **Report Lost Items** — Post details of what was lost, along with when and where.
- **Report Found Items** — Post details, including images, of items that have been found.
- **Search and Filter** — Easily search for items by category (Electronics, Documents, etc.), date, and location.
- **Secure Claiming** — Follow a structured claim process where the finder verifies the owner's identity before handing over the item.
- **Dashboard** — View a clear, consolidated feed of recently lost and found items.

3. Technology Stack — Where & How It Is Used

This section gives a simple, project-specific view of each technology — what it does in this system, not a general definition. The diagram at the end shows how one request flows through the full stack.

3.1 React.js

- Builds the pages users see — dashboard, report form, claim form, and profile.
- Fetches data from the backend and displays it as the live feed of lost/found items.
- Stores the login token after sign-in and attaches it to every request.
- Moves between pages (login → dashboard → report/claim) without reloading the browser.

3.2 Spring Boot

- Receives requests from React — login, report an item, submit a claim — through REST APIs.
- Applies the business rules, e.g. a new item starts as ACTIVE, and a claim needs an active item.
- Talks to the MySQL database to save and retrieve records.

- Sends the result back to React as a JSON response.

3.3 MySQL

- Stores all data permanently across three tables: users, items, and claims.
- Answers the queries behind each screen, e.g. fetching only ACTIVE items for the dashboard.
- Foreign keys keep claims correctly linked to the right item and user.

3.4 JWT & Spring Security

- Issues a signed token when a user logs in successfully.
- Checks this token on every request to confirm who the user is.
- Passwords are hashed before being stored, so raw passwords are never saved.

3.5 End-to-End Flow

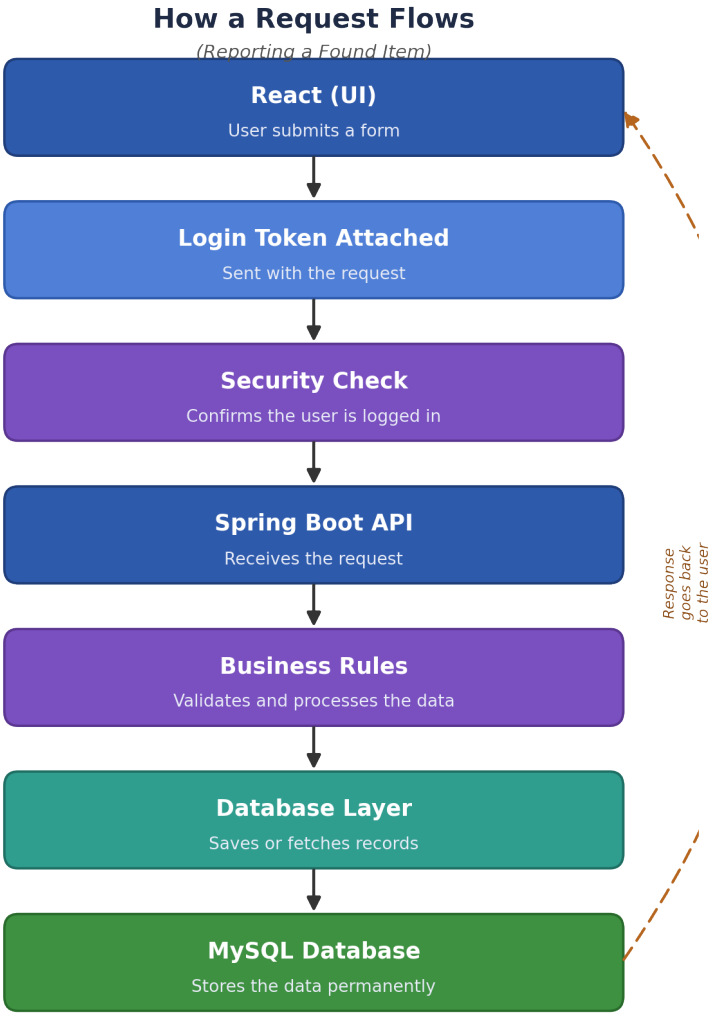


Figure — how one request moves through the stack, from the UI to the database and back.

4. Frontend Flow (React)

The frontend is built using React.js to deliver a fast, single-page application (SPA) experience. UI/UX mockups and prototyping are mapped out separately using Figma.

Core Screens

- **Landing Page / Dashboard** — Displays a feed of recent items; users toggle between "Lost" and "Found" tabs.
- **Authentication Flow** — Registration and login for students, verified using college email IDs.
- **Reporting Flow** — User clicks "Report Item" → selects type (Lost/Found) → fills out a form (title, category, location, date, image upload) → submits.
- **Claiming Flow** — User finds their lost item on the "Found" feed → clicks "Claim" → submits proof of ownership (e.g., describing a unique scratch or wallpaper) → a notification is sent to the finder.
- **Profile Page** — Users manage their reported items and track the status of their claims.

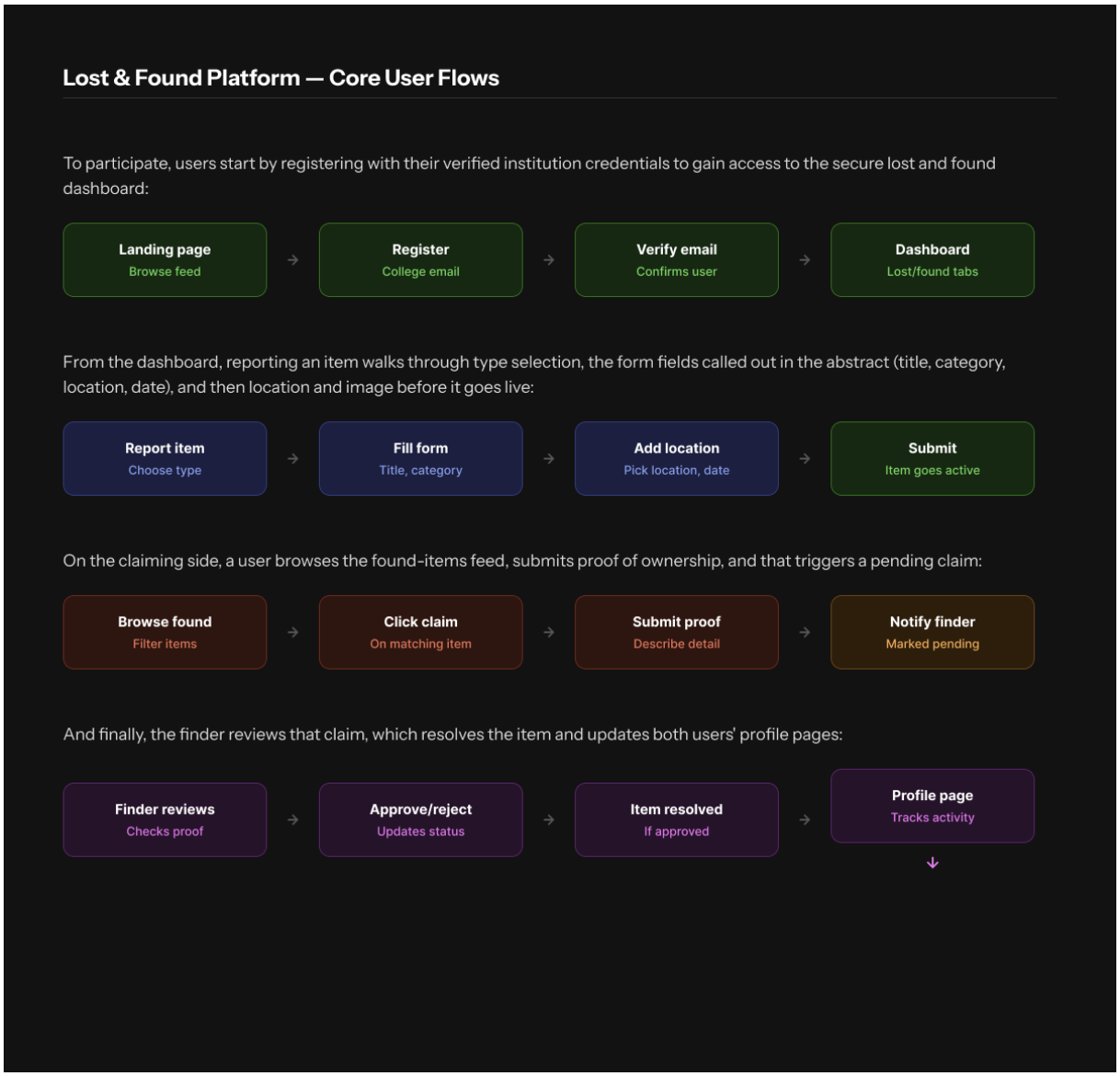


Figure 1 — Core user flows: registration, reporting, claiming, and resolution.

5. Backend Flow (Spring Boot)

The backend is a robust RESTful API built with Spring Boot. Backend workflows and API architecture are visually designed using eraser.io.

Architecture

- **Controllers** — Handle incoming HTTP requests from the React frontend.
- **Services** — Contain the core business logic, such as matching logic and claim validation.
- **Repositories** — Interface with the MySQL database using Spring Data JPA.
- **Security** — Spring Security with JWT (JSON Web Tokens) ensures only authenticated users can post or claim items.

- **Image Handling** — Found-item images are stored locally or in a cloud service, with the URL saved in the database.

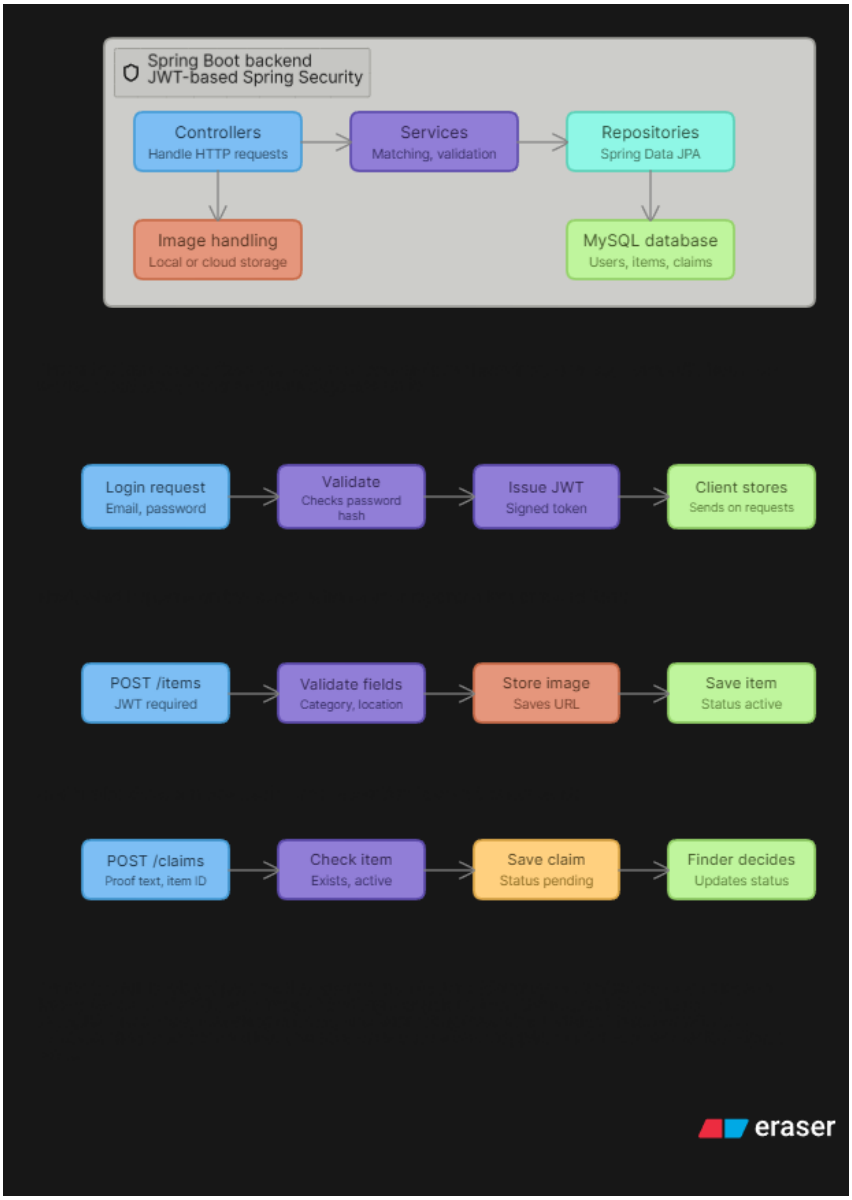


Figure 2 — Backend architecture and request flows for login, item reporting, and claims.

6. Database Schema (MySQL)

The system uses a structured relational database consisting of three tables. A user can report many items and make many claims; an item can receive many claims, though realistically only one is approved.

Users Table — one row per student or admin

Field	Type / Notes
`user_id`	Primary Key, Auto Increment
`full_name`	Varchar
`email`	Varchar, Unique — preferably college email
`password_hash`	Varchar
`phone_number`	Varchar
`role`	Enum: STUDENT, ADMIN
`created_at`	Timestamp

Items Table — one row per lost or found report

Field	Type / Notes
`item_id`	Primary Key, Auto Increment
`user_id`	Foreign Key → users.user_id (the reporter)
`title`	Varchar
`description`	Text
`item_type`	Enum: LOST, FOUND
`category`	Enum: ELECTRONICS, ID_CARD, BOOKS, KEYS, OTHER
`location`	Varchar — where it was lost / found
`date_reported`	Date
`image_url`	Varchar — optional, recommended for found items
`status`	Enum: ACTIVE, RESOLVED

Claims Table — one row per attempt to claim a found item

Field	Type / Notes
`claim_id`	Primary Key, Auto Increment
`item_id`	Foreign Key → items.item_id

`claimer_id`	Foreign Key → users.user_id (the person claiming)
`proof_description`	Text — description proving ownership
`status`	Enum: PENDING, APPROVED, REJECTED
`created_at`	Timestamp

7. Entity Relationship (ER) Diagram

The relationships below mirror the structure defined in `lost_and_found_schema.sql`: a user can report many items and make many claims, and an item can receive many claims.

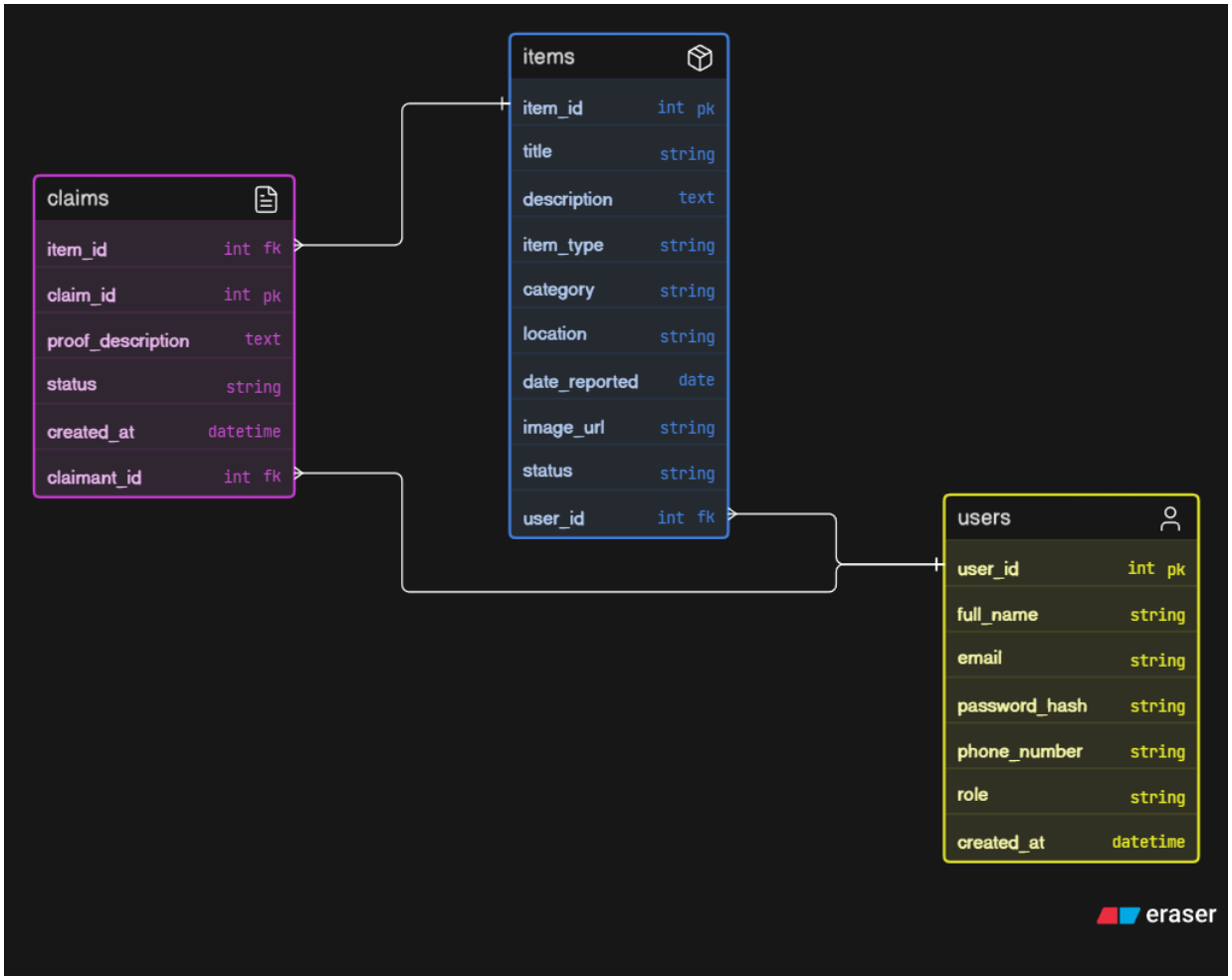


Figure 3 — Entity relationship diagram for users, items, and claims.

Relationship Summary

- **users** → **items** (one-to-many): a user can report many items.
- **users** → **claims** (one-to-many): a user can make many claims.
- **items** → **claims** (one-to-many): an item can receive many claims.